

devreport

Turn a messy project folder into a polished PDF report or slide deck

Celibistrial

October 2026

17

COMMITTS

1

AUTHOR

+2,427

LINES ADDED

1

DAY

devreport takes whatever a project left behind (dev logs, notes, a CSV, screenshots, a .zip, or a GitHub link) and turns it into a LaTeX report or Beamer deck, compiled to PDF in the browser. Plain code does the analysis. It asks three optional questions that the files can't answer: what problem the project solves, what I learned, and what comes next. Then Claude Code writes the document and compiles it.

1 *Who it's for*

It's for student builders who have to write up a hackathon, coursework or side project and would rather spend that time building. You hand it the project folder and it writes the report.

2 *How it works*

The pipeline has four stages:

1. A file upload, a zip, or a GitHub link comes in.
2. `collect.js` parses git history, logs, tables, notes and images into `data/*.csv` and `facts.json`.
3. The user's answers to the three optional questions are saved as `answers.md`.
4. `claude -p` writes `main.tex`, compiles it with `tectonic`, fixes errors, humanizes the prose and compiles again. The resulting `main.pdf` shows up inline, and the `.tex` is there to download.

The collector does all the analysis in deterministic, tested code: commits per day, work hours, most-edited files, commit types, lines per language, recurring errors (with numbers and IDs normalized so repeats group together), and column-type detection for your own CSV or JSON. Claude only writes prose. Every chart is pgfplots reading a CSV that `collect.js` computed, so the model never types a number into a chart, and stat numbers come from `facts.json`.

The server is Node's standard library and nothing else, so there's no `npm install`. `server.js` handles uploads and zip/GitHub intake, runs the collector and `claude -p`, and streams progress over SSE. The UI is a single `index.html`. The pipeline Claude follows, the tested chart snippets, the prompt and the themes all live under `.claude/skills/devreport/`.

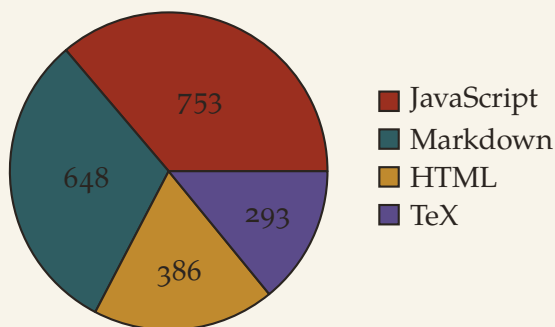


Figure 1 *Lines of code by language.*

2.1 SAFETY

Uploaded content goes to an agent, so intake is locked down:

- Claude Code runs headless with only `Read,Write,Edit,Bash(tectonic:*),Skill`, with no general shell. Uploaded content is treated as data, never as instructions.
- Zips are size-checked before extraction to catch zip bombs. Entries that escape the target path are rejected, and symlinks are deleted.
- GitHub URLs are validated and cloned without a shell, with a timeout.

2.2 THEMES

There are two themes. Paper is serif and academic, and Midnight is dark, neon and terminal-styled. Each one is a single `.sty` file that covers both reports and slides, charts included. This report uses Paper.

3 Development story

The whole project went into git on one day, across 17 commits by one author, adding 2,427 lines and removing 46. All of the commits fall in the same hour of the day.

Most commits added features. The rest were docs, tests, one fix, and a group the collector couldn't sort into a type.

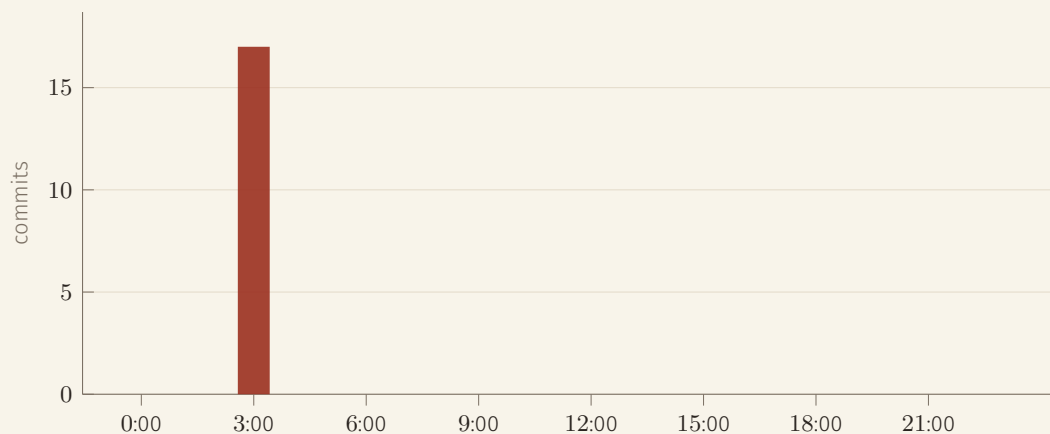


Figure 2 *Commits by hour of day.*

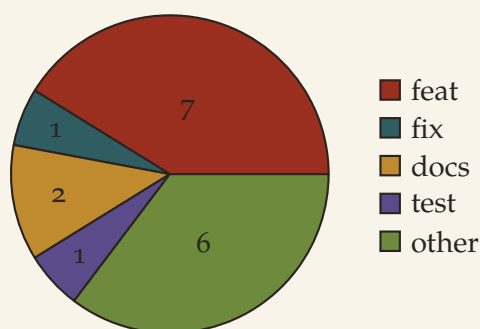


Figure 3 *Commit types.*

The files I touched most were the collector and its tests, then the two theme files, the server, the UI, the README, the skill definition and the handoff notes.

4 What works

The full pipeline runs end to end. The repo ships two sample outputs: a report in the Paper theme and a slide deck in the Midnight theme. Setup needs macOS or Linux, Node 20+, a logged-in Claude Code, tectonic, git and unzip. Then `node server.js` serves the app at `http://127.0.0.1:3000`. Each run uses your own Claude Code login and takes about 1 to 3 minutes. I chose not to host a public version, because it would run strangers' jobs on one person's subscription.

The collector has tests in `test.js`, run with `node --test test.js`. Running a bare `node --test` would also pick up tests inside cloned jobs.

4.1 KNOWN LIMITS

- It runs locally only, one job at a time.
- 16-bit PNG screenshots get re-encoded with `sips`, which only exists on macOS.
- Prompt injection is still a risk. Uploaded text reaches an agent that can write files in the job folder. The tool allowlist keeps that contained, but you should only run it on material you

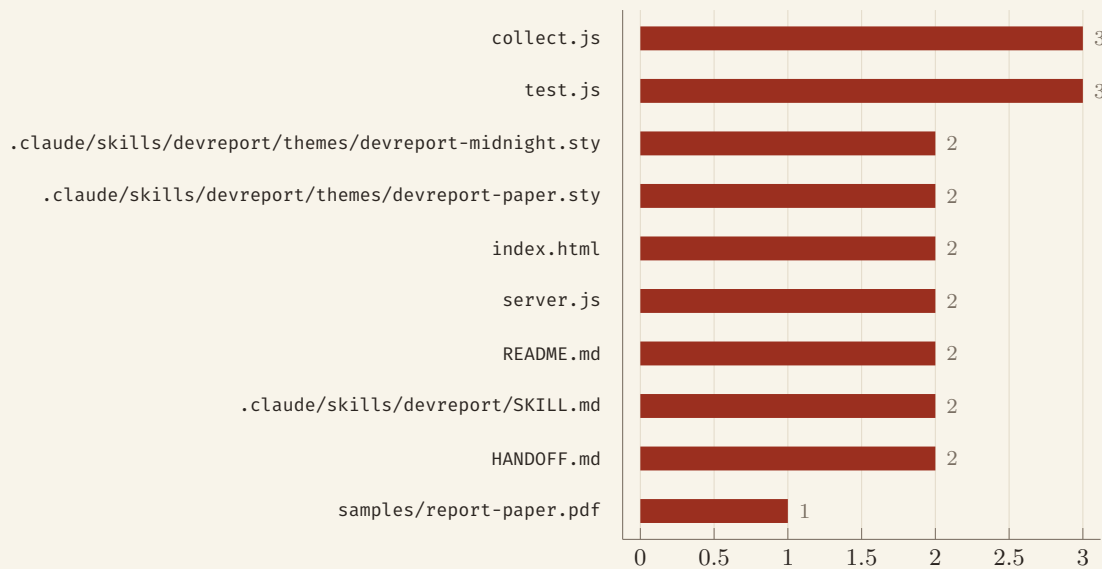


Figure 4 Most edited files.

trust.

5 AI disclosure

I built this with Claude Code as a pair programmer, and headless Claude Code is also the runtime that writes each report. The collector, intake, server and UI are ordinary code you can read and test.